CS336 Assignment 3 (scaling): Scaling Laws

Lenard Strahringer
Student ID: 06773780

Spring 2026

1 IsoFLOPs Scaling Laws (`chinchilla_isoflops`)

(a) Optimal Model Size

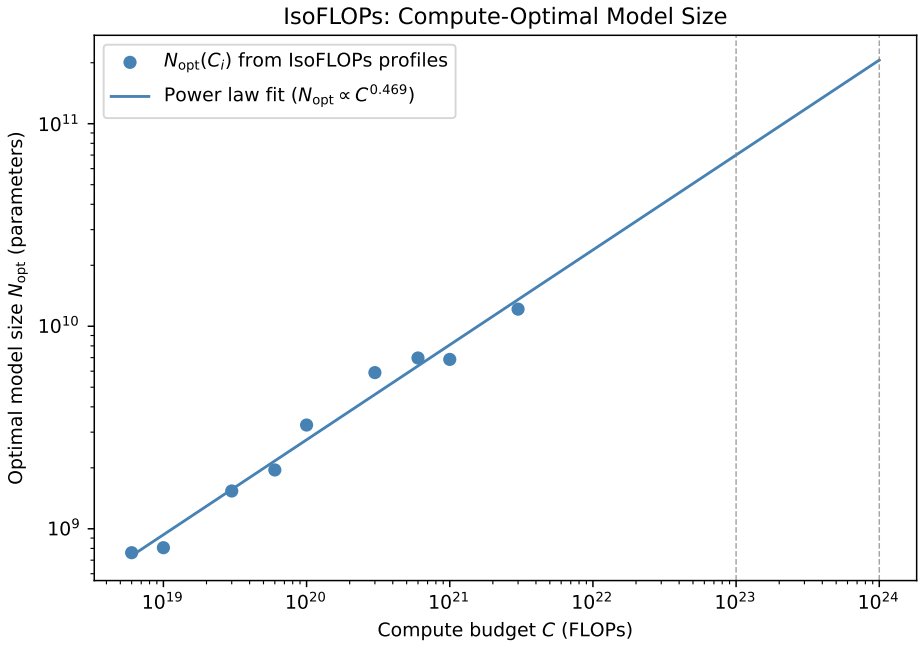

Figure 1: Best model size N_{opt} for a fixed compute budget C. Points come from IsoFLOPs profiles. The line is a power fit $N_{\text{opt}} \propto C^{a}$.

I read the fitted power law from the IsoFLOPs analysis at the two budgets of interest. At $C = 10^{23}$ FLOPs the best model size is 7.0×10^{10} parameters. At $C = 10^{24}$ FLOPs it is 2.1×10^{11} parameters.

(b) Optimal Dataset Size

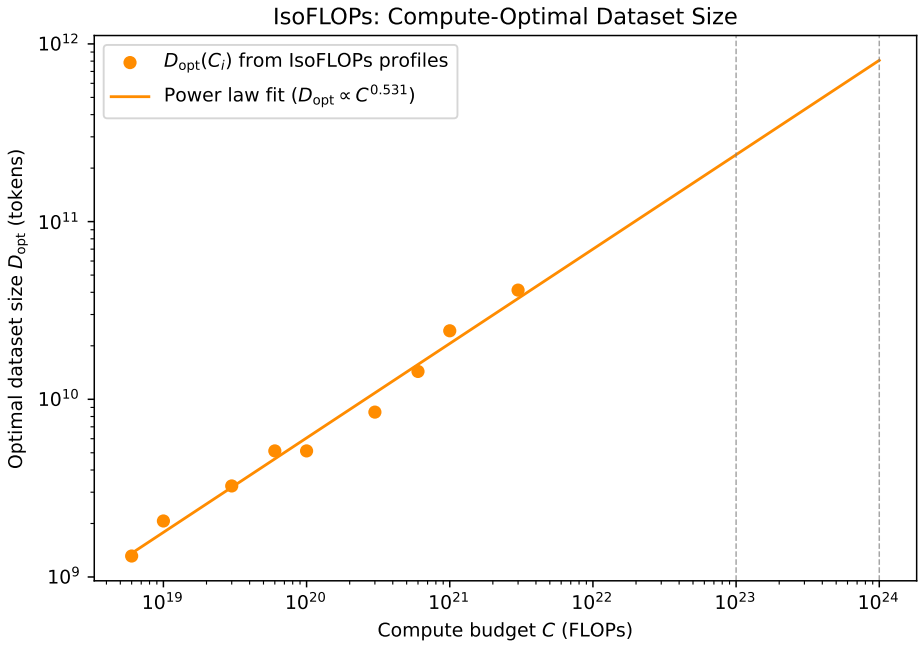

Figure 2: Best dataset size D_{opt} for a fixed compute budget C. Points come from IsoFLOPs profiles. The line is a power fit $D_{\text{opt}} \propto C^{b}$.

The same procedure for data size gives 2.4×10^{11} tokens at $C = 10^{23}$ FLOPs and 8.1×10^{11} tokens at $C = 10^{24}$ FLOPs.

2 Constructing Scaling Laws (`scaling_laws`)

2.1 What I did and why

I had a 12-hour B200 budget. I split the work into three phases so that I could tune the learning rate cheaply, then transfer it to wider models, and only then spend the bulk of the budget on points for a scaling law. The overall logic follows Kaplan et al. [1], Hoffmann et al. [2], and Yang et al. [3], but each step below states what I chose and why.

Before the main pipeline I ran one short job. I did this so that I would not waste hours on a broken API contract, a broken result path, or a wrong guess about how fast the cluster really trains (MFU). That run gave me a sanity check on all three.

For Phase 1 I needed a single peak learning rate to reuse everywhere. Tuning it on a tiny model is cheap. I used the smallest architecture the API allows: `hidden_size` = `num_attention_heads` · `head_dim` with `head_dim` = 64, which gives $d_{\text{model}} = 128$, $L = 2$, and 1.77 M non-embedding parameters. I swept nine learning rates on a log-spaced grid from 3×10^{-4} to 3×10^{-1}. Each run trained on about 16 M tokens with a cosine schedule matched to that length, as in Hoffmann et al. [2] (Approach 3). I set the top of the sweep high enough that the largest rate would diverge. If the best value sat on the upper edge, I would not know whether a higher rate could be better; an interior optimum is easier to trust.

Phase 2 had to tell me how to scale that rate when d_{model} grows. Yang et al. [3] justify μP, where the optimal rate is width-invariant, but the full setup is not available through the assignment API. I therefore used the simpler width scaling they and Hoffmann et al. [2] discuss,

$$\eta(d) = \eta_{\text{proxy}} \cdot \left(\frac{d_{\text{proxy}}}{d} \right)^{\gamma}, \tag{1}$$

and I estimated γ from data instead of fixing it to the common guess $\gamma = 1$. I ran small grids of three rates at $d_{\text{model}} \in \{320, 512\}$, centered on the prediction from $\gamma = 1$. Both first grids put the winner at the low end ($\times 0.5 / \times 1 / \times 2$ relative to a center), so I ran a second pass at $\times 0.25$ and $\times 0.5$ around the previous best. I stopped once both widths had an interior best, because then the estimate of γ is not pinned to a grid edge.

For Phase 3 I froze η_{proxy} from Phase 1 and γ from Phase 2. I trained six model families from 1.77 M to 116 M non-embedding parameters at compute targets $C \in \{10^{18}, 3 \times 10^{18}, 10^{19}\}$ FLOPs; the exact pairing of family and C is in Table 1. I kept $d_{\text{model}}/L \approx 50$ because Kaplan et al. [1] find that aspect ratio moves results only weakly at fixed parameter count, so I did not want to spend budget on exotic shapes. I also made sure that at 10^{18} and 3×10^{18} FLOPs at least four families appear. Hoffmann et al. [2] (Approach 2) use IsoFLOPs cuts to check a parametric fit; with several families at the same C I can make the same kind of comparison.

(d_{model}, L)	N (non-embedding)	Compute levels (FLOPs)
$(192, 4)$	1.77 M	$\{1\times10^{18}\}$
$(320, 6)$	7.37 M	$\{1\times10^{18}, 3\times10^{18}\}$
$(448, 9)$	21.7 M	$\{1\times10^{18}, 3\times10^{18}, 1\times10^{19}\}$
$(576, 10)$	39.8 M	$\{1\times10^{18}, 3\times10^{18}, 1\times10^{19}\}$
$(704, 12)$	71.5 M	$\{3\times10^{18}, 1\times10^{19}\}$
$(832, 14)$	116.4 M	$\{1\times10^{18}, 3\times10^{18}\}$

Table 1: Phase 3 model families and per-family compute levels in my sweep.

The hosted API reserves `max_runtime_seconds` up front and refunds what I do not use. I set each job's cap from `safety_factor`$\cdot C/($`MFU`$\cdot$`peak_FLOPs`$)$ with `MFU` $= 0.4$ and `safety_factor` $= 1.5$, clipped at two hours, so that planned training had headroom against slowdowns. I still submitted Phase 3 in three waves (five, five, three jobs) so refunds from finished runs returned budget before I queued the next wave. That order was a practical way to stay inside the 12-hour pool without hand-tuning every reservation.

2.2 Phase 1 and Phase 2 outcomes

Phase 1 peaked at $\eta_{\text{proxy}} = 2.25 \times 10^{-2}$ with validation loss 5.85. The endpoints 3×10^{-4} and 3×10^{-1} landed at 7.98 and 6.68, so the best point is clearly inside the sweep.

Phase 2 gave best rates 4.50×10^{-3} at $d_{\text{model}} = 320$ and 2.81×10^{-3} at $d_{\text{model}} = 512$. Averaging the implied γ from both widths,

$$\gamma \;=\; \frac{1}{2} \sum_{d \in \{320,\, 512\}} \frac{\log\big(\eta(d)/\eta_{\text{proxy}}\big)}{\log(d_{\text{proxy}}/d)} \;=\; 1.628. \tag{2}$$

That is steeper than $\gamma = 1$, but Yang et al. [3] (Figure 1) already show that this simple rule can move the best rate by nearly an order of magnitude when width grows, so I did not treat $\gamma = 1$ as a hard truth. I carried $\gamma = 1.628$ into Phase 3.

2.3 Phase 3, salvage, and the anchor check

Every Phase 3 run hit my wall-time cap. From eval throughput I inferred MFU between 1.6% and 4.7%, far below the 40% I assumed when sizing `max_runtime`. Three of the smallest jobs never logged an eval block; ten others returned one to three partial validation losses.

Dropping those runs would leave almost no data, so I kept them and estimated trained tokens as $D_{\text{actual}} = (\texttt{evals\_completed}/16) \cdot D_{\text{planned}}$ for each job. I fit the law on $(N, D_{\text{actual}}, L_{\text{partial}})$. The worry is upward bias: if training stops while the cosine schedule is still near the peak rate, loss can look worse than for a run that decays on schedule.

I tested that worry on $(d_{\text{model}}, L) = (576, 10)$. I re-ran training with `total_train_tokens` set to the same D_{actual} as the truncated job so the cosine could finish (final rate $0.1\,\eta_{\text{peak}}$). The truncated partial log gave $L = 3.7820$; the matched schedule gave $L = 3.7813$. The gap is 0.001 nats, so at this (N, D) the salvage choice does not materially shift the loss. I therefore kept D_{actual} for the fit.

2.4 Fitting the scaling law

I followed Hoffmann et al. [2] Approach 3 (Equations 2–3),

$$\widehat{L}(N, D) \;=\; E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}}. \tag{3}$$

I fit (E, A, B, α, β) by minimizing Huber loss on log-residuals with $\delta = 10^{-3}$, matching Hoffmann et al., using L-BFGS-B. Non-convex fits can land in local minima, so I repeated the fit from eight starting points covering $\log E \in \{\log 1, \log 2\}$, $\alpha \in \{0.3, 0.5\}$, and $\beta \in \{0.3, 0.5\}$, and I kept the solution with the lowest objective.

On the ten salvaged Phase 3 points,

$$E = 3.376, \quad A = 9.66 \times 10^{6}, \quad B = 9.64 \times 10^{6}, \quad \alpha = 1.049, \quad \beta = 0.857.$$

My α and β exceed Hoffmann et al. [2]'s Chinchilla values ($\alpha = 0.34$, $\beta = 0.28$), but the ratio $\alpha/\beta = 1.22$ matches theirs (1.21) almost exactly. Hoffmann et al. [2] Equation 5 shows that the compute-optimal split between N and D depends only on α/β, so my prediction for how to divide compute between model and data stays aligned with Chinchilla even though the raw exponents differ. The narrower N span in my data (5.4× versus 230× in Hoffmann et al.) is the natural reason the individual exponents look sharper.

2.5 Quality of fit

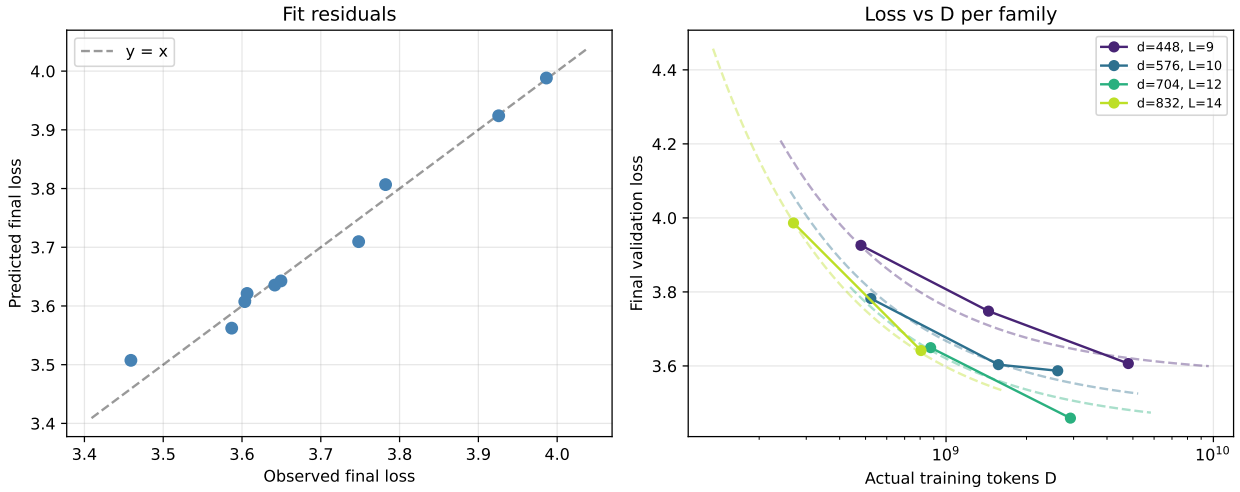

Figure 3: Left: predicted vs. observed loss for my ten salvaged Phase 3 runs. Right: loss vs. tokens by model family; dashed lines are the fit at fixed N.

Figure 3 shows that residuals stay small across the ten points ($\max|r| = 0.048$ nats, RMSE 0.025 nats), so the parametric form describes my measured range well. I would still be cautious far outside it. My parameter counts only run from 21.7 M to 116 M (5.4×), and Hoffmann et al. [2] warn that exponent estimates move a lot when the fit range is narrow. The observed D/N ratios fall in $[12, 220]$ with most runs above the Chinchilla-optimal $D/N \approx 20$, so the surface may be less reliable if I extrapolate to very different data-to-parameter ratios.

2.6 Prediction for 48 B200-hours and why it changed

The unconstrained Hoffmann et al. [2] recipe at 7.78×10^{20} FLOPs (48 B200-hours at peak) predicts $N^* = 1.23 \times 10^9$ and $D^* = 1.05 \times 10^{11}$. I could not use that plan: at the MFU I actually saw ($\approx 5\%$), finishing that training would need on the order of 211 wall-clock hours, which breaks a 48-hour cap.

I therefore minimized $\widehat{L}(N, D)$ subject to $6ND = M \cdot \texttt{peak\_FLOPs} \cdot 48\,\text{h}$. I set $M = 0.05$ from my largest completed runs (116 M parameters) because that is the only hard evidence I had; larger models often improve MFU on tensor cores, but guessing higher without measurements felt optimistic. I then shaved D by 20% below the constrained optimum to leave slack (≈ 9.6 hours at $M = 0.05$) so the cosine schedule can finish even if throughput is a bit worse than expected.

Table 2 lists the configuration. The implied validation loss from the fit is

$$L^* = \widehat{L}(3.13 \times 10^8, 1.66 \times 10^{10}) = 3.4051. \tag{4}$$

2.7 Hyperparameters for the leaderboard run

I set depth and width in the spirit of Kaplan et al. [1]. I matched Hoffmann et al. [2] on training mechanics: cosine to the full token count, 10× LR decay, AdamW with $\beta_2 = 0.95$. Peak learning rate comes from Yang et al. [3] through the width scaling with my measured $\gamma = 1.628$, giving $\eta = 6.90 \times 10^{-4}$ from $\eta_{\text{proxy}} = 2.25 \times 10^{-2}$ at $d_{\text{proxy}} = 128$.

Hyperparameter	Value
`num_hidden_layers`	22
`hidden_size`	1088
`num_attention_heads`	17
`head_dim`	64
`intermediate_size`	3072
`total_train_tokens`	1.66×10^{10}
`train_batch_size`	128
Peak learning rate η	6.90×10^{-4}
Warmup fraction	0.05
Final-LR fraction	0.10 (cosine to $0.1\,\eta$)
AdamW (β_1, β_2)	(0.9, 0.95)
Weight decay	0.01
Gradient clip norm	1.0

Table 2: Suggested hyperparameters for my 48 B200-hour run. Non-embedding parameters $N = 3.13 \times 10^8$. The model satisfies `hidden_size` = `num_attention_heads` · `head_dim` and `num_key_value_heads` = `num_attention_heads`.

References

[1] J. Kaplan *et al.*, "Scaling Laws for Neural Language Models." 2020.

[2] J. Hoffmann *et al.*, "Training Compute-Optimal Large Language Models." 2022.

[3] G. Yang *et al.*, "Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer." 2022.